A ROS 2 Digital Twin for Construction Robot Inspection

Tian Tan
Department of Civil Engineering
The University of Hong Kong
Hong Kong, China
tiantan05@connect.hku.hk

Abstract

Construction inspection still relies on manual site walks, visual judgement, and records that are often separated from the actual work front. Mobile robots can make inspection more repeatable, but construction sites are cluttered, changing, and safety-critical, so autonomy should first be tested in a controlled digital environment. This paper presents a ROS 2-based construction robot digital twin prototype for an indoor inspection task. The system combines Gazebo simulation, a four-wheel mobile robot model, LiDAR and RGB sensing, Nav2 autonomous navigation, SLAM-ready mapping, mission orchestration, and a browser-based FastAPI dashboard. The implemented task is to start from point A, navigate through a site-like room, approach a wall-mounted AprilTag target at point B, and report mission state and telemetry to an operator. The codebase was reviewed at the package and node level to document the implemented architecture rather than only the final demonstration. The prototype connects simulation, navigation, telemetry, and human supervision into an executable construction robot twin. Its limits are also clear: the AprilTag is currently a known map target rather than a detected visual pose, the platform is a mobile inspection robot rather than a mobile manipulator, and the work has not been deployed on physical hardware.

Keywords: construction digital twin; construction robotics; ROS 2; Gazebo; Nav2; mobile robot inspection

1 Introduction

Construction projects generate decisions in places where information is incomplete and conditions change quickly. A route that is clear in the morning can become blocked by material stacks in the afternoon. Temporary fences and no-go zones are moved as work fronts progress. Inspection staff need to confirm progress, safety conditions, and asset state, but manual site walks are time-consuming and hard to repeat at high frequency. This is one reason mobile robots are attractive for construction monitoring: they can collect data along planned routes and return structured state information to an operator.

The difficulty is that construction sites are not designed for robots. Floors may be uneven, obstacles are irregular, work zones are shared with people, and the relevant information is not only geometric. A robot must know where it is, where it is allowed to move, what it is trying to

inspect, and whether its current behaviour is safe. These requirements make a digital twin more useful than a simple animation. A useful construction robot twin should combine a virtual site, a robot model, sensor streams, navigation logic, mission state, and a decision interface.

This project implements that idea at prototype scale. It contains a ROS 2 workspace with separate packages for robot description, simulation, navigation, mission logic, and a local web dashboard. The robot is a four-wheel mobile inspection platform with LiDAR, camera, odometry, transforms, and a pan/tilt gimbal. The site is a Gazebo room with walls, material-like obstacles, barriers, a tool chest, an inspection column, and a wall-mounted AprilTag. The task is deliberately narrow: the robot starts at point A, navigates to the AprilTag approach point B, and reports the mission as complete.

The work is not presented as a new navigation algorithm. Its value is in the integration: a small but reproducible digital twin architecture for construction robot inspection, with simulation, navigation, mission logic, telemetry, and operator feedback working together. The discussion also keeps the scope explicit. Although the CIVIL 8025 Project 4 direction mentions mobile manipulation and MoveIt 2 as possible elements, this implementation is a mobile inspection robot twin and does not include an arm, MoveIt 2 planning, or real AprilTag pose estimation.

2 Literature Review

2.1 Construction digital twins

Digital twin research in construction has moved beyond the idea of a 3D model. Boje, Guerriero, Kubicki, and Rezgui (2020) argue that a semantic construction digital twin should connect model information, data streams, and reasoning so that the digital representation can support decisions. This distinction matters here because a Gazebo model alone would not be enough. The project becomes closer to a twin when the simulated robot state, sensor data, mission events, and dashboard controls are connected through ROS 2 and exposed to the user.

Jiang, Ma, Broyd, and Chen (2021) differentiate digital twins from BIM and cyber-physical systems in civil engineering. Their review emphasizes that a digital twin needs a relationship between an as-is physical or simulated counterpart, a virtual model, and a connection that updates state. The present prototype is simulation-first, so it should be understood as a construction robot twin prototype or digital shadow rather than a field-synchronized operational twin. This framing is important because it avoids overstating what the implementation proves.

Madubuike, Anumba, and Khallaf (2022) review construction digital twin applications and show that the construction sector still lags behind more controlled industries. The barriers include fragmented data, heterogeneous processes, and difficulty in linking models to real-time decision workflows. Those barriers are visible even in a small robot project. The robot model, map, mission waypoints, telemetry, and web interface all exist in different files and packages; the engineering value comes from making these parts communicate.

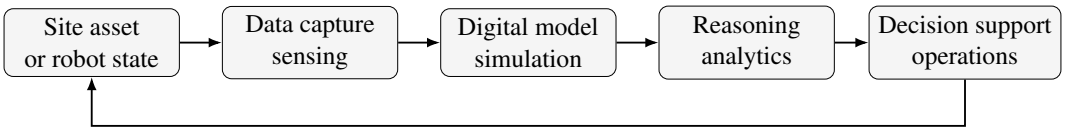


Figure 1: Digital twin literature commonly frames the twin as a closed loop from asset state and sensing to model-based reasoning and operational decisions; the structure is synthesized from Boje et al. (2020), Jiang et al. (2021), and Madubuike et al. (2022).

2.2 Construction robotics and inspection

Construction automation has long promised productivity and safety gains, but adoption is slowed by project-specific site conditions. Bock (2015) describes construction robotics as a field moving toward broader application while still facing technical and organizational limits. Davila Delgado et al. (2019) classify robotics and automated systems in construction and highlight adoption challenges such as cost, training, site variability, and unclear integration with existing workflows. These points explain why a small inspection robot twin is a reasonable course project: it tests integration and supervision before attempting physical deployment.

Mobile robots are particularly relevant for progress monitoring and inspection. Asadi et al. (2018) present a vision-based mobile robotic system for real-time construction applications and emphasize the need for autonomous navigation, sensing, mapping, and real-time processing. Their work is more advanced than this prototype, but the system requirements are similar. A construction inspection robot needs to navigate, observe, and report. It also needs an operator interface because construction engineers are unlikely to interact only through terminal commands.

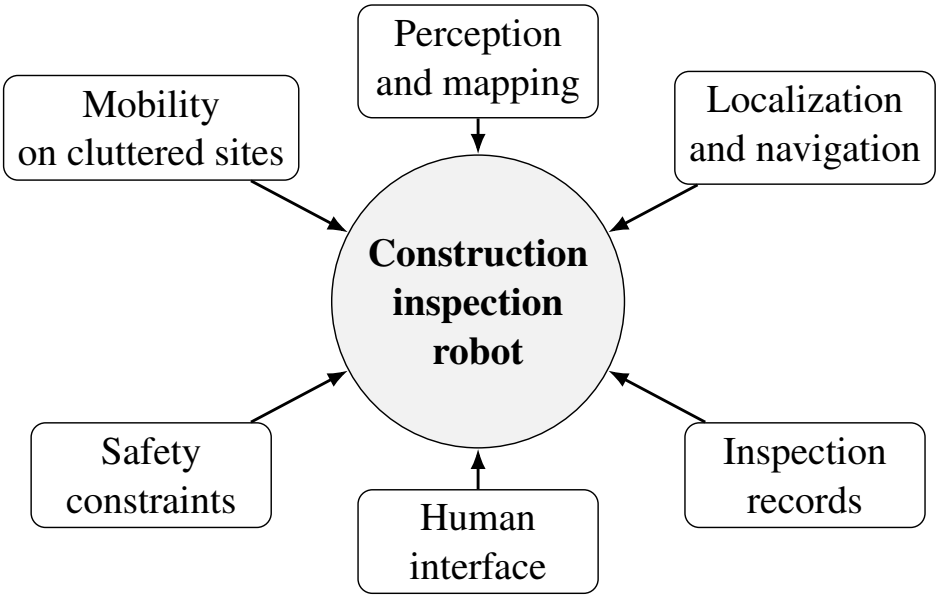


Figure 2: Construction-robotics studies identify recurring requirements around mobility, perception, navigation, safety, interfaces, and data records; the diagram summarizes themes reviewed by Davila Delgado et al. (2019) and Asadi et al. (2018).

Location and tracking research also supports this direction. Teizer, Venugopal, and Walia (2008) discuss real-time location sensing for workforce, equipment, and materials, connecting tracking to construction logistics and safety decisions. The present project uses simulated odometry, LiDAR, and map-frame transforms rather than UWB, but the same decision principle applies: robot position and environment state only become useful when they are made observable and actionable.

2.3 Navigation, SLAM, and open robot software

ROS was introduced as an open-source robot software framework to support modular robot development and reuse (Quigley et al., 2009). ROS 2 continues that idea with a communication layer suitable for distributed nodes, actions, transforms, sensors, and lifecycle-managed systems (Open Robotics, n.d.a). This is why the project separates robot description, navigation, mission orchestration, telemetry, and web bridge responsibilities instead of writing one monolithic script.

For autonomous mobile robots, navigation is the operational core. The Nav2 stack provides localization, planning, control, costmaps, behavior trees, and recovery behaviours for ROS 2 robots (Macenski, Martin, White, & Clavero, 2020; Open Navigation LLC, n.d.). This project uses Nav2's `NavigateToPose` action and costmap pipeline rather than writing a custom path planner. That design choice is appropriate for a digital twin prototype because the research question is system integration in a construction context, not low-level planner invention.

SLAM is also relevant because construction layouts change. Kohlbrecher, Meyer, von Stryk, and Klingauf (2011) present a flexible SLAM system that supports mapping from laser scans. In this implementation, SLAM Toolbox is included in a separate mapping launch workflow. The saved map is then loaded by Nav2 for normal navigation. This separation mirrors a practical construction workflow: map the environment, save a usable representation, then run inspection missions against that representation.

2.4 Human supervision and telemetry

Construction robotics should not be treated as full autonomy without supervision. The site engineer needs to know whether the robot is waiting, navigating, blocked, or finished. The digital twin therefore needs a telemetry layer. In this project, telemetry is not only a visual dashboard; it is a structured JSON payload containing pose, velocity, command velocity, nearest obstacle, gimbal joints, safety warnings, target metadata, and mission events. This makes the system closer to an engineering decision-support tool than a simple simulation.

The literature also points toward semantic and context-aware navigation. Karimi, Braga, Iordanova, and St-Onge (2021) propose using building information to support safer and more meaningful robot navigation on construction sites. This project does not yet connect BIM or IFC data, but the waypoint file and marker node are a small step in that direction: the robot does not navigate to arbitrary coordinates only; it navigates to named inspection points and target metadata that can be displayed to the operator.

3 Methodology

3.1 System architecture

The code was reviewed by reading the ROS packages, launch files, configuration files, mission nodes, web backend, and frontend dashboard. The implementation is organized into five main subsystems. The robot description package defines the URDF/Xacro robot, collisions, inertias, sensors, and gimbal controllers. The simulation package launches Gazebo and the indoor construction-like world, following the same ROS 2/Gazebo integration pattern described in the official simulation documentation (Open Robotics, n.d.b). The navigation package contains Nav2 parameters, the static map, SLAM configuration, RViz configuration, and waypoint definitions. The mission package implements readiness checks, autonomous goal dispatch, manual control, telemetry, video recording support, and RViz markers. The web folder implements the FastAPI bridge and dashboard.

Figure 3 uses the same architecture diagram as the project presentation. The architecture is intentionally bidirectional at the operator layer: the dashboard displays telemetry, but it also sends commands back to the mission and manual control topics.

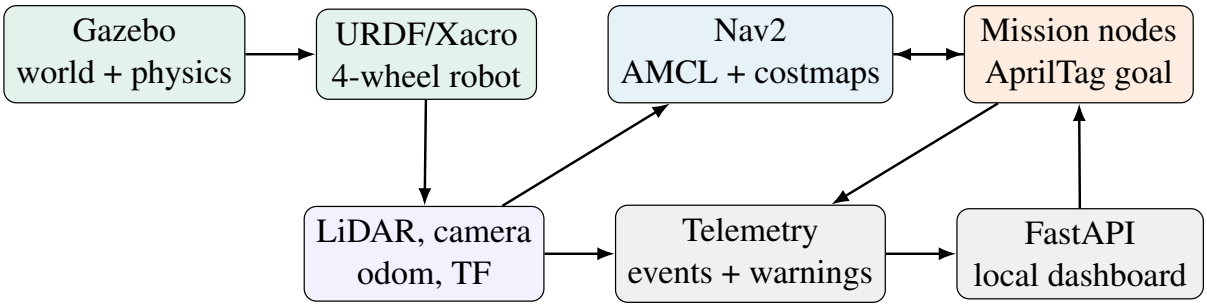


Figure 3: Digital twin architecture linking simulation, navigation, mission logic, telemetry, and operator interface.

3.2 Robot, site, and sensing model

The robot is a compact four-wheel mobile inspection platform. The rear wheels are driven by a Gazebo differential-drive plugin and publish odometry. The model includes collision geometry and inertial values for the chassis, wheels, mast, camera mount, LiDAR, and camera. A pan/tilt camera gimbal is controlled through a ROS 2 control position controller. The robot publishes a LiDAR scan on `/scan`, camera images on `/camera/image_raw`, odometry on `/odom`, and transforms through the standard TF tree.

The URDF source was checked in `src/my_robot_description/urdf/robot.urdf.xacro`. It appears to be a custom course-project model built from primitive URDF geometry rather than a downloaded mesh-based robot. The file defines the `orbit_camera_bot` base, four wheel links, mast, pan/tilt camera joints, LiDAR frame, camera frame, Gazebo material settings, and ROS 2 control interfaces. The model is intentionally simple: the rear wheels are actuated through the Gazebo differential-drive plugin, while the front wheels support the chassis. This is sufficient for the inspection-navigation objective without claiming mobile manipulation.

The Gazebo world represents a small site-like interior rather than an empty laboratory. It includes walls, collision obstacles, construction barriers, material-like blocks, a rebar-style object, a tool chest, an inspection column, and a wall-mounted AprilTag. These details matter because Nav2 costmaps and LiDAR scans need obstacles to reason about. The AprilTag is visually present in the world, but the current autonomous completion logic uses known map coordinates from `waypoints.yaml`.

3.3 Navigation and mission execution

The navigation workflow starts with the robot spawned at point A. In the waypoint file, A is defined at $x = -3.8$ m and $y = -3.6$ m, while the target approach point B is defined at $x = -1.5$ m and $y = 3.65$ m with a yaw of 1.5708 rad. The wall AprilTag target is stored separately at $x = -1.5$ m, $y = 4.93$ m, and $z = 1.05$ m with a completion radius of 1.4 m. This distinction is useful: the robot should not collide with the wall or tag surface, so it navigates to an approach pose rather than the tag position itself.

Nav2 is configured with AMCL, a static map, global and local costmaps, obstacle and inflation layers, a DWB local planner, Navfn global planner, velocity smoothing, and lifecycle management. The robot radius is set to 0.31 m. The local costmap uses a 4 m rolling window and a 0.5 m inflation radius, while the global costmap uses a 0.6 m inflation radius. Linear speed is limited to 0.35 m/s and angular speed to 1.2 rad/s. These values are simple but explicit safety constraints.

The mission orchestrator waits for the Nav2 action server and the `map` to `base_link` transform before sending a goal. It publishes compact event strings including readiness, navigation

start, acceptance, success, failure status, and AprilTag reached events. To avoid the previous line-overflow issue, the report describes these event names in prose rather than placing a long unbreakable sequence of `texttt` tokens in one line.

3.4 Telemetry, markers, and dashboard

The telemetry node subscribes to mission state, events, odometry, commanded velocity, LiDAR, joint states, orbit status, and video status. It publishes a JSON payload at 5 Hz. The dashboard bridge exposes this payload through HTTP and WebSocket endpoints and converts camera images into a JPEG stream. The frontend displays the mission state, route points, camera stream, manual drive controls, gimbal sliders, pose, velocity, command velocity, nearest obstacle distance, AprilTag metadata, safety warnings, and event log.

RViz is used as the engineering visualization layer. The waypoint marker node publishes waypoint arrows, point labels, target cylinder markers, and AprilTag target markers. The RViz configuration displays the grid, map, robot model, TF, laser scan, mission markers, and camera. This complements the browser dashboard: the dashboard is better for operation, while RViz is better for debugging the robot state in the map frame.

Table 1: Implemented modules and evidence found in the project files.

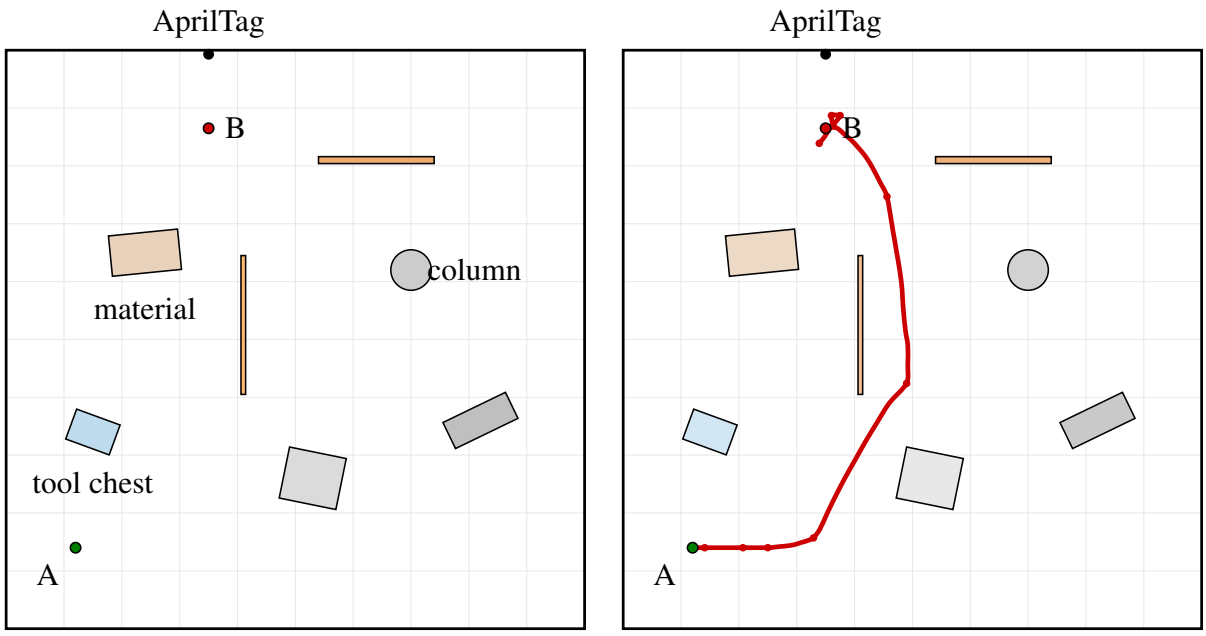
Module	Implemented evidence
Robot description	URDF/Xacro collision geometry, inertias, wheel joints, mast, LiDAR, RGB camera, pan/tilt gimbal, Gazebo plugins, and ROS 2 control joints.
Simulation	Gazebo launch file, indoor room world, construction-like obstacles, inspection column asset, and wall AprilTag target.
Navigation	Nav2 bringup, AMCL initial pose, static map, SLAM Toolbox mapping launch, costmaps, DWB local planner, Navfn planner, velocity smoother, and RViz config.
Mission layer	Start/stop/reset commands, Nav2 readiness check, `NavigateToPose` dispatch, completion events, waypoint markers, manual drive limits, and gimbal control.
Web twin	FastAPI ROS bridge, WebSocket telemetry stream, HTTP command endpoints, camera stream, dashboard controls, metrics, warnings, and event display.

4 Results and Discussion

4.1 Digital twin demonstration

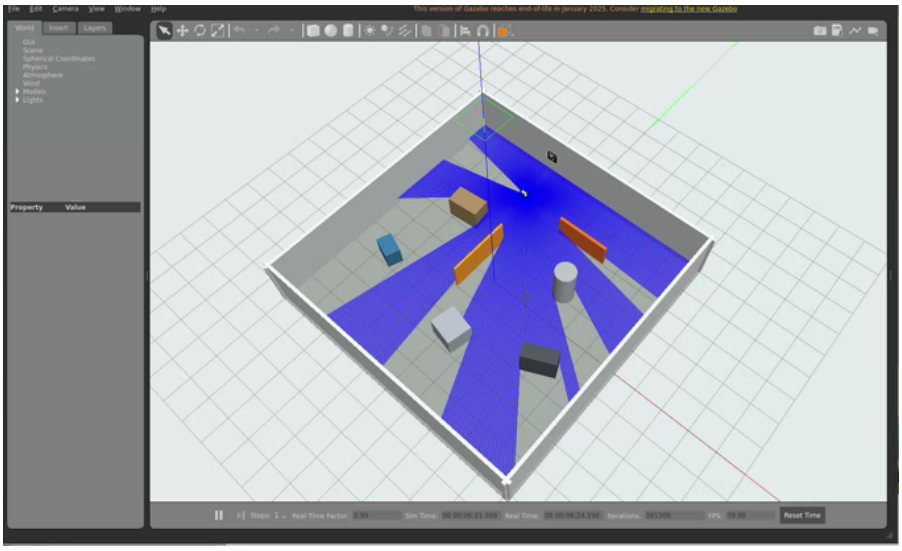
The prototype completed the intended demonstration workflow: a simulated robot was launched in a construction-like room, localized on a saved map, commanded from a web dashboard, navigated toward the AprilTag approach point, and displayed state through both the dashboard and RViz. Figure 4 shows the corrected 2D site layout, the recorded A–B mission trajectory, and the Gazebo world during the demonstration. Figure 5 adds the RViz frame extracted from the same video, showing the robot, map, scan, TF, and mission markers.

The route is short, but the integration is non-trivial. The system must bring up Gazebo, spawn



(a) Corrected 2D site layout redrawn from `indoor_room.world` poses.

(b) Recorded A–B mission trajectory.



(c) Gazebo world screenshot from the recorded demonstration.

Figure 4: Navigation evidence used by the digital twin. The 2D obstacles follow the Gazebo world poses, and the trajectory shows the recorded A–B mission path.

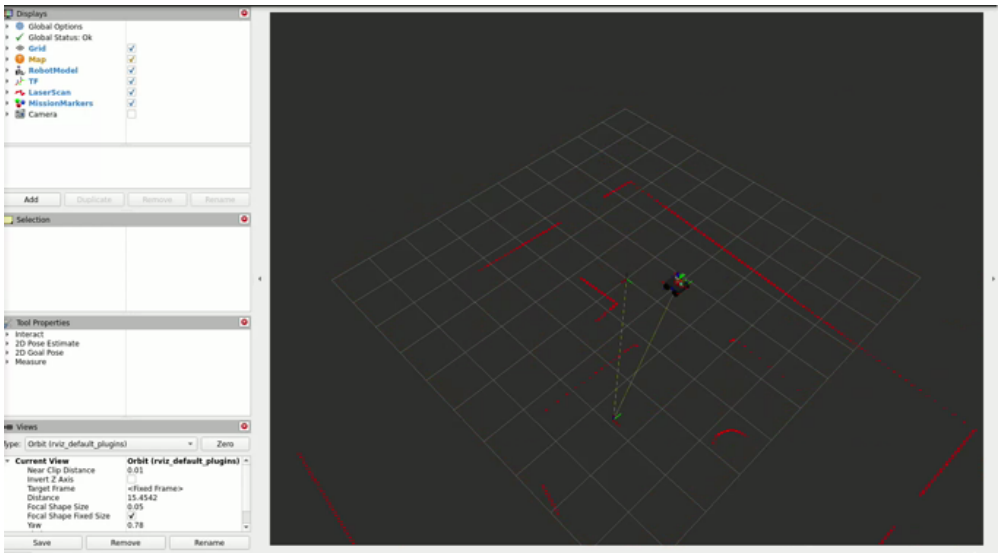


Figure 5: RViz snapshot extracted from the recorded mission video, showing map-frame visualization of the robot, scan, TF, and mission markers.

the URDF robot, publish robot state, activate controllers, run Nav2 lifecycle nodes, publish the initial pose, serve the dashboard, and coordinate the mission command. If any piece is missing, the mission node reports a readiness failure rather than silently sending a goal. This is important for construction use because engineering demonstrations often fail at integration boundaries, not at one isolated algorithm.

4.2 Operator observability

The dashboard screenshot in Figure 6 shows why the web interface is useful. A user can see the mission state, route metadata, camera stream, pose, velocity, nearest obstacle, AprilTag target, safety panel, and event log without reading ROS topics manually. The dashboard does not replace RViz, but it creates an operator-facing view that is closer to what a construction engineer would need during inspection.

The telemetry design also creates a path for future data logging. Since the node publishes structured JSON, the same payload could be stored in a time-series database, connected to a project dashboard, or used for post-mission analysis. In the current course prototype, the data are shown live rather than archived, but the interface design is already compatible with a more persistent digital twin workflow.

4.3 Construction relevance

The project is relevant to construction because it represents the robot as part of a site operation rather than as a generic indoor robot. The target is a wall-mounted inspection object. The map contains barriers, material-like objects, and a column. Safety is represented through collision geometry, costmap inflation, speed limits, obstacle warnings, and an immediate stop command. These details are small, but they move the prototype toward construction engineering concerns: where the robot can move, what it is inspecting, and what the operator can observe.

At the same time, the prototype should not be interpreted as a deployable site robot. Real construction sites have dust, lighting variation, temporary occlusions, reflective materials, uneven surfaces, people, communication issues, and regulatory constraints. The simulated LiDAR and camera are cleaner than real sensors. The map is known and small. The AprilTag is not detected

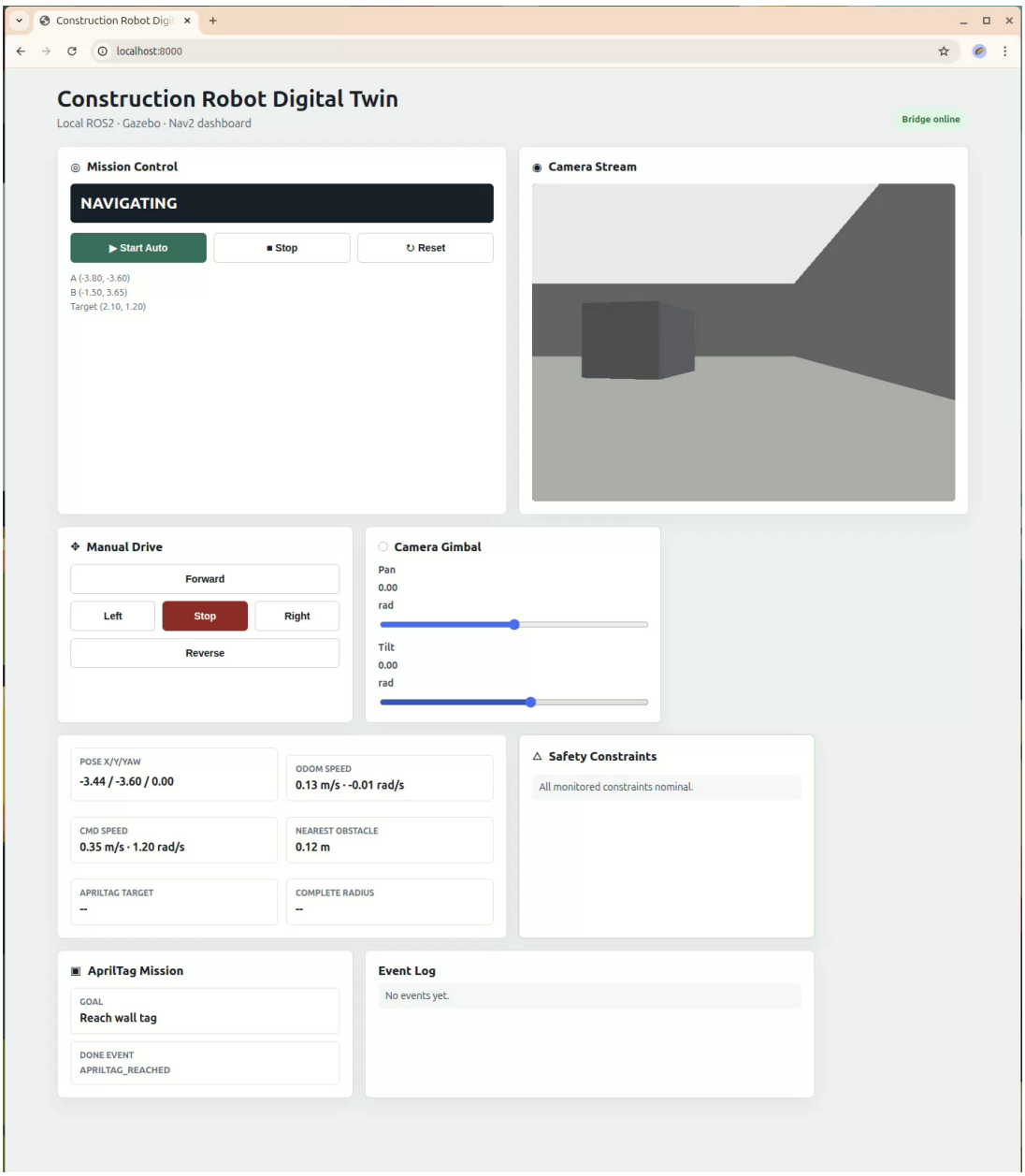


Figure 6: Local web dashboard for mission commands, manual control, camera stream, telemetry, warnings, and events.

from images. Therefore, the correct conclusion is that the project demonstrates an architecture and workflow, not field readiness.

4.4 Limitations and future work

Four limitations stand out. First, the AprilTag target is not used for visual pose estimation. The mission currently navigates to a known map coordinate and checks proximity to the stored tag position. Second, there is no manipulator arm or MoveIt 2 planning, so the prototype is not a mobile manipulator. Third, the web dashboard does not yet display the costmap or LiDAR scan directly, so RViz remains necessary for debugging. Fourth, the system has not been tested with real robot hardware.

Future work should first close the loop between visual perception and navigation. AprilTag detection could estimate the final tag pose and support a short visual-servoing approach. The dashboard could display a simplified map, scan, and costmap overlay. If the project is later extended into mobile manipulation, MoveIt 2 planning would need to be added with construction obstacles represented as collision objects. Physical deployment would also require calibration, sensor noise handling, emergency stop hardware, and safety procedures for human-robot coexistence.

5 Conclusions

This paper presented a ROS 2 construction robot digital twin prototype for mobile inspection. The project integrates Gazebo simulation, a robot model, Nav2 navigation, SLAM-ready mapping, mission logic, telemetry, RViz visualization, and a web dashboard. The system is grounded in construction digital twin literature because it links a virtual site, robot state, mission events, sensor streams, and operator decisions. It is also grounded in construction robotics literature because it focuses on inspection, navigation, sensing, and supervision rather than only visual animation.

The main result is an executable and observable prototype. The robot can be commanded from the dashboard, navigates from point A to the AprilTag approach point B, and reports state through ROS topics, telemetry JSON, RViz markers, and browser UI elements. The main engineering value is the clean separation of packages and the explicit handling of readiness, safety constraints, and telemetry. The main limitation is scope: this is a simulated mobile inspection robot twin, not a field-deployed mobile manipulator. Future work should add real AprilTag detection, richer dashboard map visualization, and physical validation before any stronger claim is made.

Acknowledgements

The author acknowledges the CIVIL 8025 teaching team at The University of Hong Kong for the project direction and course context. The implementation builds on open-source ROS 2, Gazebo, Nav2, SLAM Toolbox, FastAPI, and related robotics tools.

References

Asadi, K., Ramshankar, H., Pullagurla, H., Bhandare, A., Shanbhag, S., Mehta, P., … Wu,

T. (2018). Vision-based integrated mobile robotic system for real-time applications in construction. *Automation in Construction*, *96*, 470–482. Retrieved from https://doi.org/10.1016/j.autcon.2018.10.009 doi: 10.1016/j.autcon.2018.10.009

Bock, T. (2015). The future of construction automation: Technological disruption and the upcoming ubiquity of robotics. *Automation in Construction*, *59*, 113–121. Retrieved from https://doi.org/10.1016/j.autcon.2015.07.022 doi: 10.1016/j.autcon.2015.07.022

Boje, C., Guerriero, A., Kubicki, S., & Rezgui, Y. (2020). Towards a semantic construction digital twin: Directions for future research. *Automation in Construction*, *114*, 103179. Retrieved from https://doi.org/10.1016/j.autcon.2020.103179 doi: 10.1016/j.autcon.2020.103179

Davila Delgado, J. M., Oyedele, L., Ajayi, A., Akanbi, L., Akinade, O., Bilal, M., & Owolabi, H. (2019). Robotics and automated systems in construction: Understanding industry-specific challenges for adoption. *Journal of Building Engineering*, *26*, 100868. Retrieved from https://doi.org/10.1016/j.jobe.2019.100868 doi: 10.1016/j.jobe.2019.100868

Jiang, F., Ma, L., Broyd, T., & Chen, K. (2021). Digital twin and its implementations in the civil engineering sector. *Automation in Construction*, *130*, 103838. Retrieved from https://doi.org/10.1016/j.autcon.2021.103838 doi: 10.1016/j.autcon.2021.103838

Karimi, S., Braga, R. G., Iordanova, I., & St-Onge, D. (2021). Semantic optimal robot navigation using building information on construction sites. In *Proceedings of the 38th international symposium on automation and robotics in construction* (pp. 57–64). Retrieved from https://doi.org/10.22260/ISARC2021/0010 doi: 10.22260/ISARC2021/0010

Kohlbrecher, S., Meyer, J., von Stryk, O., & Klingauf, U. (2011). A flexible and scalable SLAM system with full 3d motion estimation. In *2011 ieee international symposium on safety, security, and rescue robotics* (pp. 155–160). Retrieved from https://doi.org/10.1109/SSRR.2011.6106777 doi: 10.1109/SSRR.2011.6106777

Macenski, S., Martin, F., White, R., & Clavero, J. G. (2020). *The marathon 2: A navigation system.* arXiv:2003.00368. Retrieved from https://arxiv.org/abs/2003.00368

Madubuike, O. C., Anumba, C. J., & Khallaf, R. (2022). A review of digital twin applications in construction. *Journal of Information Technology in Construction*, *27*, 145–172. Retrieved from https://www.itcon.org/papers/2022_08-ITcon-Madubuike.pdf doi: 10.36680/j.itcon.2022.008

Open Navigation LLC. (n.d.). *Nav2 Navigation Stack: Humble Documentation.* https://api.nav2.org/nav2-humble/html/md_doc_README.html. (Accessed May 2026)

Open Robotics. (n.d.a). *ROS 2 Documentation: Humble Hawksbill.* https://docs.ros.org/en/humble/. (Accessed May 2026)

Open Robotics. (n.d.b). *Setting up a robot simulation: Gazebo.* https://docs.ros.org/en/humble/Tutorials/Advanced/Simulators/Gazebo/Gazebo.html. (Accessed May 2026)

Quigley, M., Conley, K., Gerkey, B., Faust, J., Foote, T., Leibs, J., ... Ng, A. Y. (2009). ROS: An open-source robot operating system. In *Icra workshop on open source software.* Retrieved from https://ai.stanford.edu/~mquigley/papers/icra2009-ros.pdf

Teizer, J., Venugopal, M., & Walia, A. (2008). Ultrawideband for automated real-time three-dimensional location sensing for workforce, equipment, and material positioning and tracking. *Transportation Research Record*, *2081*(1), 56–64. Retrieved from https://doi.org/10.3141/2081-06 doi: 10.3141/2081-06